API & Command Reference Guide

Author: Mohoshin Islam Tonmoy (Roll: BSSE1750)

1. Interactive CLI Engine (Client Commands)

The client terminal operates an interactive loop driven by `ReadFile()` in a dedicated thread. Commands are parsed and processed locally before any network transmission occurs. Below is the reference table for supported terminal commands.

Command	Syntax	Description
Users	`/users`	Requests and displays a real-time list of all authenticated users currently connected to the relay server.
Private Message	`/msg <username> <text>`	Initiates the End-to-End Encrypted (E2EE) routing protocol. Encrypts the payload via XOR stream cipher and triggers an audio bell alert (`\a`) on the recipient's terminal upon delivery.
Help	`/help`	Prints the local CLI manual, detailing available commands and current cryptographic session status.
Clear Display	`/clear`	Clears the terminal scrollback buffer utilizing VT100 ANSI escape sequences, restoring the prompt layout.
Terminate	`/exit`	Executes a graceful shutdown sequence. Releases local memory buffers, closes the Win32 Winsock descriptors, and terminates active thread handles.

> **Note on Parsing:** The routing engine intercepts the `/msg` header. The server parses only the destination `<username>` for socket mapping but treats the rest of the payload strictly as an opaque byte array.

2. Custom Zero-Abstraction API (libc-Free Utilities)

To assert complete control over memory bounds and avoid standard library overhead, the system implements proprietary low-level C functions to replace standard `string.h` utilities.

Function Prototype	Implementation Detail & Purpose
`void* my_memset(void* dest, int val, size_t count);`	Replaces standard `memset`. Performs byte-level pointer arithmetic to initialize socket buffers and clear sensitive cryptographic variables (like Diffie-Hellman secrets) from RAM.
`size_t my_strlen(const char* str);`	Replaces standard `strlen`. Computes payload length iteratively up to the null terminator, crucial for determining the exact boundary of ciphertext arrays prior to socket dispatch.
`char* my_strcpy(char* dest, const char* src);`	Replaces standard `strcpy`. Copies buffer contents manually, utilized during terminal UI string formatting and packet header construction.

3. Network Socket API Mapping (Win32)

The system interacts directly with the Windows networking stack via `ws2_32.lib`. The following core Winsock2 primitives are utilized:

```
// Initialization WSAStartup(MAKEWORD(2, 2), &wsaData); // Non-blocking Socket
Creation SOCKET relay_socket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP); //
Concurrency Engine HANDLE hThread = CreateThread(NULL, 0, ClientHandlerThread,
(LPVOID)client_socket, 0, &dwThreadId);
```